



Súper Ahorro

Swift vs Kotlin — Tecnologías Móviles

ALUMNOS

Martínez Jesus Manuel
Bulatovich Juan Cruz

MATERIA

Tecnologías Móviles · Prof. Fanin

INSTITUCIÓN

UNDEF — Fuerza Aérea Argentina

PLATAFORMA

iOS 17+ · Swift 5.9 · Xcode 16

STACK TECNOLÓGICO

SwiftUI

UI declarativa — como Jetpack Compose

SwiftData

ORM local — como Room

@Observable

ViewModel reactivo — como LiveData

Supabase

Backend PostgreSQL + Auth + Storage

Swift Charts

Gráficos — como MPAndroidChart

Vision OCR

Escaneo de tickets con cámara

Face ID

Biometría — como BiometricPrompt

WidgetKit

Widget en pantalla de inicio

Variables, Tipos y Null Safety

✗ PROBLEMA EN JAVA / SIN PATRÓN

Java permite asignar null a cualquier variable.
Resultado: NullPointerException en runtime — el 'error del billón de dólares'.

✓ QUÉ RESUELVE

Kotlin y Swift hacen los nulls explícitos con tipos opcionales. Si compilas, es seguro. El compilador te obliga a manejar el caso null.

◆ KOTLIN / ANDROID

```
// val = inmutable | var = mutable
val supermercado: String = "Carrefour"
var total: Double = 0.0
var isSynced: Boolean = false

// Null Safety — tipos nullable
var ticketURL: String? = null
val largo = ticketURL?.length // safe call
val url = ticketURL ?: "sin-url" // Elvis
```

◆ SWIFT — Súper Ahorro

```
// let = inmutable | var = mutable
// Models/Compra.swift
var id: UUID
var supermercado: String
var total: Double
var isSynced: Bool = false

// Optional — mismo concepto que nullable
var ticketURL: String? = nil
let url = ticketURL ?? "sin-url" // ??=Elvis
guard let uid = currentUserID else {
    throw NSError.noSession // falla rápida
}
```

Modelos de Datos — @Model (SwiftData)

✗ PROBLEMA EN JAVA / SIN PATRÓN

Sin ORM, guardar un objeto en una base de datos requiere escribir SQL manualmente: CREATE TABLE, INSERT INTO, etc. Muy verboso y frágil.

✓ QUÉ RESUELVE

Room (@Entity) y SwiftData (@Model) mapean objetos Swift/Kotlin a tablas SQL automáticamente. Escribís la clase, el ORM maneja el SQL.

◆ KOTLIN / ANDROID

```
@Entity(tableName = "compras")
data class Compra(
    @PrimaryKey
    val id: String = UUID.randomUUID().toString(),
    val supermercado: String,
    var total: Double = 0.0,
    val ticketURL: String? = null,
    @ForeignKey(entity=Producto::class,
        onDelete=CASCADE)
    val productos: List<Producto> = listOf()
)
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/Models/Compra.swift
import SwiftData

@Model // equivalente a @Entity de Room
final class Compra {
    var id: UUID
    var userId: String
    var supermercado: String
    var total: Double
    var ticketURL: String? = nil
    var isSynced: Bool = false
    @Relationship(deleteRule: .cascade)
    var productos: [Producto] // FK cascade
}
```

Activity vs SwiftUI View + Ciclo de vida

✗ PROBLEMA EN JAVA / SIN PATRÓN

En Android, el desarrollador debe gestionar 7 estados del ciclo de vida (onCreate, onResume, onStop...). Fácil olvidar actualizar datos al volver a una pantalla.

✓ QUÉ RESUELVE

SwiftUI elimina el ciclo de vida explícito. La vista se reconstruye sola cuando cambia el estado. `.onAppear` y `@Query` reemplazan el ciclo de vida.

◆ KOTLIN / ANDROID

```
class HomeActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_home)
        viewModel.compras.observe(this) { updateUI(it) }
    }
    override fun onResume() {
        super.onResume()
        viewModel.cargarCompras() // recargar datos
    }
    override fun onDestroy() {
        binding.recyclerView.adapter = null // leak
    }
}
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/Views/Compras/HomeView.swift
struct HomeView: View {
    // @Query = LiveData<List<T>> - auto-observable
    @Query(sort: \Compra.fecha, order: .reverse)
    private var todasLasCompras: [Compra]

    var body: some View {
        ScrollView { /* UI aquí */ }
        // equivalente a onResume()
        .onAppear {
            verificarPresupuesto()
            writeWidgetData()
        }
    }
}
```

ViewModel + Estado Reactivo

✗ PROBLEMA EN JAVA / SIN PATRÓN

Mezclar UI + lógica de negocio + red + BD en una Activity/View crea un 'God Object' — imposible de mantener, testear o reusar.

✓ QUÉ RESUELVE

ViewModel separa la lógica de la UI. La Vista solo observa el estado y llama métodos. El ViewModel nunca importa SwiftUI ni Activity.

◆ KOTLIN / ANDROID

```
class ExpenseViewModel : ViewModel() {
    private val _isLoading =
        MutableLiveData<Boolean>(false)
    val isLoading: LiveData<Boolean> = _isLoading

    fun login() {
        viewModelScope.launch {
            _isLoading.value = true
            repository.login(email, password)
            _isLoading.value = false
        }
    }
}
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/ViewModels/AuthViewModel.swift
import Observation // sin import SwiftUI

@Observable // ← sin @Published, sin LiveData
final class AuthViewModel {
    var isLoading: Bool = false
    var errorMessage: String?

    func login() async {
        isLoading = true
        defer { isLoading = false }
        do {
            try await supabase.login(email, password)
        } catch { errorMessage = "..."}
    }
}
```


Coroutines → async/await

✗ PROBLEMA EN JAVA / SIN PATRÓN

Las operaciones de red o BD en el hilo principal bloquean la UI — la app se congela. Con callbacks se crea 'callback hell': código anidado ilegible.

✓ QUÉ RESUELVE

Coroutines (Kotlin) y async/await (Swift) permiten escribir código asíncrono de forma SECUENCIAL y legible, sin bloquear el UI thread.

◆ KOTLIN / ANDROID

```
// suspend fun = puede suspenderse sin bloquear
suspend fun fetchCompras(): List<Compra>

// viewModelScope.launch = Task { } en Swift
fun sincronizarPendientes() {
    viewModelScope.launch {
        val pendientes = dao.getSinSinc()
        for (c in pendientes) {
            api.upsertCompra(c) // suspend
            c.isSynced = true
        }
    }
}
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/Services/SyncService.swift
@MainActor // = Dispatchers.Main para todo
final class SyncService {
    func sincronizarPendientes(
        context: ModelContext) async {
        let descriptor = FetchDescriptor<Compra>(
            predicate: #Predicate { !$0.isSynced })
        let pendientes = try? context.fetch(descriptor)
        for compra in pendientes ?? [] {
            try await SupabaseService.shared
                .upsertCompra(id: compra.id, ...)
            compra.isSynced = true // main thread
        }
    }
}
```

Networking — Supabase SDK (URLSession)

❌ PROBLEMA EN JAVA / SIN PATRÓN

Las llamadas de red en el hilo principal congela la UI y causan ANR (Android Not Responding) o freeze en iOS. El código de red mezclado con la UI es imposible de mantener.

✅ QUÉ RESUELVE

Retrofit (Android) y Supabase SDK/URLSession (iOS) abstraen HTTP. `async/await` asegura que corra en background. `Codable` decodifica JSON automáticamente.

◆ KOTLIN / ANDROID

```
// Interfaz de API con Retrofit
interface SupabaseApi {
    @GET("compras")
    suspend fun getCompras(
        @Query("order") order: String
    ): Response<List<CompraDTO>>
    @POST("compras")
    suspend fun crearCompra(
        @Body compra: CompraDTO
    ): Response<CompraDTO>
}
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/Services/SupabaseService.swift
// Codable = Gson de Kotlin (decodifica JSON)
private struct CompraRemota: Codable {
    let id: UUID
    let supermercado: String
    let total: Double
    let ticketUrl: String?
    enum CodingKeys: String, CodingKey {
        case ticketUrl = "ticket_url" // snake_case
    }
}

func fetchCompras() async throws -> [CompraDTO] {
    try await client.from("compras")
        .select().order("fecha").execute().value
}
```

Permisos + Biometría

✗ PROBLEMA EN JAVA / SIN PATRÓN

Sin sistema de permisos, cualquier app podría acceder a cámara, contactos o biometría sin que el usuario lo sepa. Riesgo de privacidad crítico.

✓ QUÉ RESUELVE

AndroidManifest.xml e Info.plist declaran qué capacidades usa la app. El sistema informa al usuario y solicita confirmación. Sin permiso, la operación falla.

◆ KOTLIN / ANDROID

```
<!-- AndroidManifest.xml -->
<uses-permission
    android:name="android.permission.CAMERA" />
<uses-permission
    android:name="android.permission.USE_BIOMETRIC" />

// Kotlin — solicitar en runtime
val request = registerForActivityResult(
    RequestPermission()
) { isGranted ->
    if (isGranted) abrirCamara()
    else mostrarRacional()
}
```

◆ SWIFT — Súper Ahorro

```
// Info.plist — equivalente al AndroidManifest
NSCameraUsageDescription:
    "Súper Ahorro escanea tickets..."
NSFaceIDUsageDescription:
    "Face ID protege tus datos"

// BiometricService.swift — LocalAuthentication
func authenticate(reason: String) async -> Bool {
    let ctx = LAContext()
    // iOS muestra Face ID automáticamente
    return try await ctx.evaluatePolicy(
        .deviceOwnerAuthenticationWithBiometrics,
        localizedReason: reason
    )
}
```


UI Declarativa — Compose → SwiftUI + Swift Charts

✗ PROBLEMA EN JAVA / SIN PATRÓN

La UI imperativa (XML + Views) requiere actualizar componentes manualmente: `textView.text='...'`, `progressBar.visibility=GONE`. Fácil quedar en estado inconsistente.

✓ QUÉ RESUELVE

Compose y SwiftUI describen QUÉ mostrar, no CÓMO actualizar. Cuando cambia el estado, la UI se reconstruye automáticamente. Cero actualizaciones manuales.

◆ KOTLIN / ANDROID

```
@Composable
fun GastoItem(compra: Compra) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Text(compra.supermercado,
                style=MaterialTheme.typography
                    .titleMedium)
            Text("${%.2f".format(compra.total)},
                style=MaterialTheme.typography
                    .headlineMedium)
        }
    }
}
```

◆ SWIFT — Súper Ahorro

```
// MisGastos/Views/Estadisticas/EstadisticasView.swift
import Charts

// BarMark = gráfico de barras
Chart(gastosPorSuper, id: \.nombre) { item in
    BarMark(
        x: .value("Tienda", item.nombre),
        y: .value("Total", item.total)
    )
    .foregroundStyle(Color.saGreen.gradient)
}
// SectorMark = donut chart
SectorMark(angle: .value("$", item.total),
            innerRadius: .ratio(0.55))
```

Navegación: Intent → NavigationStack

ESTRUCTURA DE NAVEGACIÓN — Súper Ahorro

```
MisGastosApp (@main)
├─ AppLockWrapper (Face ID lock)
│   └─ SplashScreen
│       ├── LoginView      (si no hay sesión)
│       └─ MainTabView     (TabView 5 tabs)
│           ├── Tab 1: NavigationStack
│           │   ├── HomeView
│           │   │   └─ NuevaCompraView (sheet)
│           │   └─ Tab 2: NavigationStack
│           │       ├── HistorialView
│           │       │   └─ DetalleCompraView
│           │       │       └─ EditarCompraView (sheet)
│           │   └─ Tab 3: EstadisticasView
│           │   └─ Tab 4: ComparativaView
│           └─ Tab 5: PerfilView
```

ANDROID — Intent (clase)

```
// Navegar con Intent
val intent = Intent(
    this,
    DetalleActivity::class.java
).apply {
    putExtra("compra_id", id)
}
startActivity(intent)

// Recibir dato
val id =
intent.getStringExtra("compra_id")
viewModel.cargarCompra(id)
```

SWIFT — NavigationLink (real)

```
// MainTabView.swift
TabView(selection: $selectedTab) {
    NavigationStack { HomeView() }
    .tag(0)
    .tabItem {
        Label("Inicio",
              systemImage:
"house.fill")
    }
}
// HomeView.swift — navegar
NavigationLink {
    DetalleCompraView(compra:
compra)
    // objeto directo, sin putExtra
} label: { recentRow(compra:
compra) }
```

COMPARTIR — Intent.ACTION_SEND → ShareLink

```
// DetalleCompraView.swift
// Equivalente exacto a Intent(ACTION_SEND)
ShareLink(item: resumenTexto) {
    Label("Compartir",
          systemImage: "square.and.arrow.up")
}
```

Los conceptos son universales.

● Null Safety

Kotlin y Swift eliminan el NullPointerException de Java

● MVVM

ViewModel separa UI de lógica — en Android y iOS

● UI declarativa

Compose y SwiftUI usan el mismo paradigma

● async/await

Coroutines y Swift Concurrency resuelven lo mismo

● ORM

Room y SwiftData son el mismo patrón con distinta API

● Permisos

AndroidManifest ↔ Info.plist — mismo sistema, distinta sintaxis

Súper Ahorro

📱 iOS 17+ · Swift 5.9 · SwiftUI

📄 SwiftData + Supabase

🔒 Face ID · OCR · WidgetKit

📊 Swift Charts (3 tipos)

github.com/Yisus538/MisGastos

com.undef.superahorro.MartinezBulatovich